

Network Traffic Anomaly Detection using the UNSW-NB15 Dataset

Artificial Intelligence in IT Security Systems – Home Assignment

Student: Iordan Alexandru Razvan

Professor: Madalina Doinea Zurini

Academic year: 2025–2026

Introduction

This project investigates the application of machine learning techniques for network traffic anomaly detection using the UNSW-NB15 dataset. The primary objective is to distinguish between normal and malicious network traffic by comparing unsupervised anomaly detection methods, supervised classification models, and a deep learning-based autoencoder approach. Such techniques are highly relevant in modern cybersecurity systems, where attack patterns evolve rapidly and labeled data is often limited.

Data Pre-processing and Exploration

The UNSW-NB15 dataset consists of network traffic flows labeled as either normal or belonging to specific attack categories. The training set contains 175,341 samples, while the test set contains 82,332 samples. Each record includes 39 numerical features and 3 categorical features (proto, service, and state), in addition to the binary label and the multi-class attack category.

Exploratory data analysis revealed that many numerical features exhibit strong skewness and heavy-tailed distributions, which is typical for network traffic data. Several features also show high variance and the presence of outliers.

Prior to model training, the following pre-processing steps were applied:

- i. categorical features were transformed using one-hot encoding
- ii. numerical features were standardized to zero mean and unit variance
- iii. class imbalance was considered, with SMOTE applied only for linear supervised models

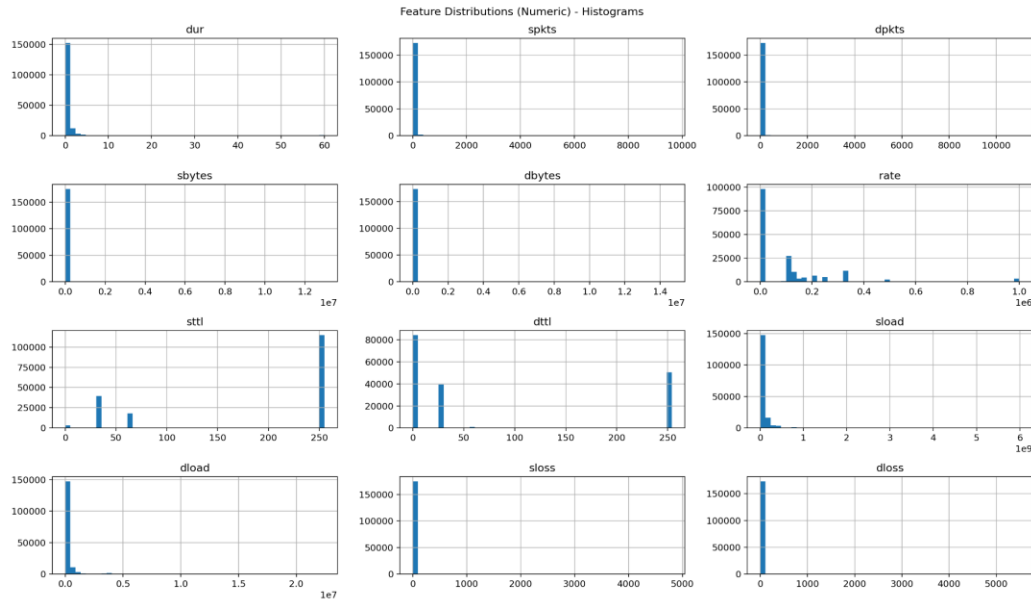


Figure 1: Distribution of selected numerical features (histograms)

Figure 2 presents boxplots of selected numerical features. The distributions exhibit strong skewness and heavy-tailed behavior, with several features spanning multiple orders of magnitude. Traffic load features (e.g., sload) dominate the scale, causing other features to appear compressed. This observation highlights the necessity of feature scaling prior to model training and reflects the presence of significant outliers typical of network traffic data.

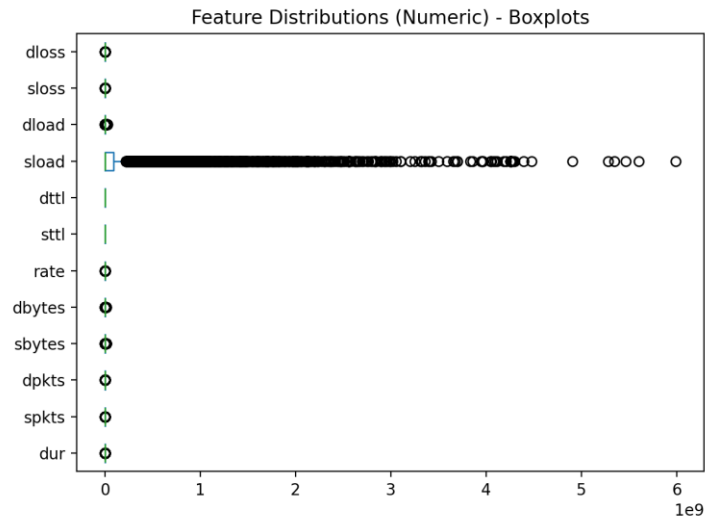


Figure 2: Boxplots of selected numerical features, highlighting outliers

Figure 3 shows the boxplots of the numerical features after applying standardization. After scaling, all features are brought to a comparable range, which makes it easier to analyze and use them in machine learning models. Some features have very narrow distributions because they are mostly binary or take only a few values, while others such as traffic rate or load show a much wider spread. This happens because network traffic is often bursty, and standardization does not limit or clip extreme values, so unusually high activity can appear several standard deviations away from the mean. Overall, this figure confirms that feature scaling was necessary and helps explain how different types of network features behave before being used for anomaly detection.

Some features go beyond ± 1 because scaling doesn't cap values. Network traffic can spike a lot, and those spikes remain visible after standardization.

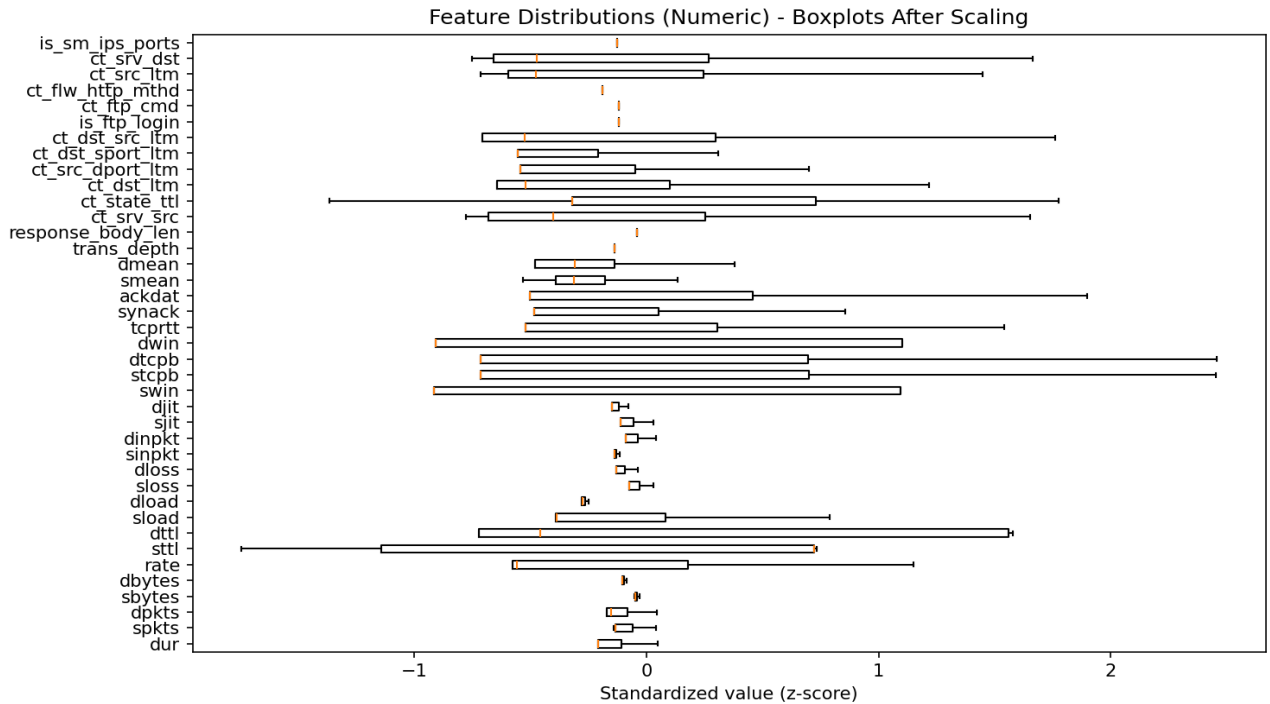


Figure 3: Boxplots of selected numerical features after scaling

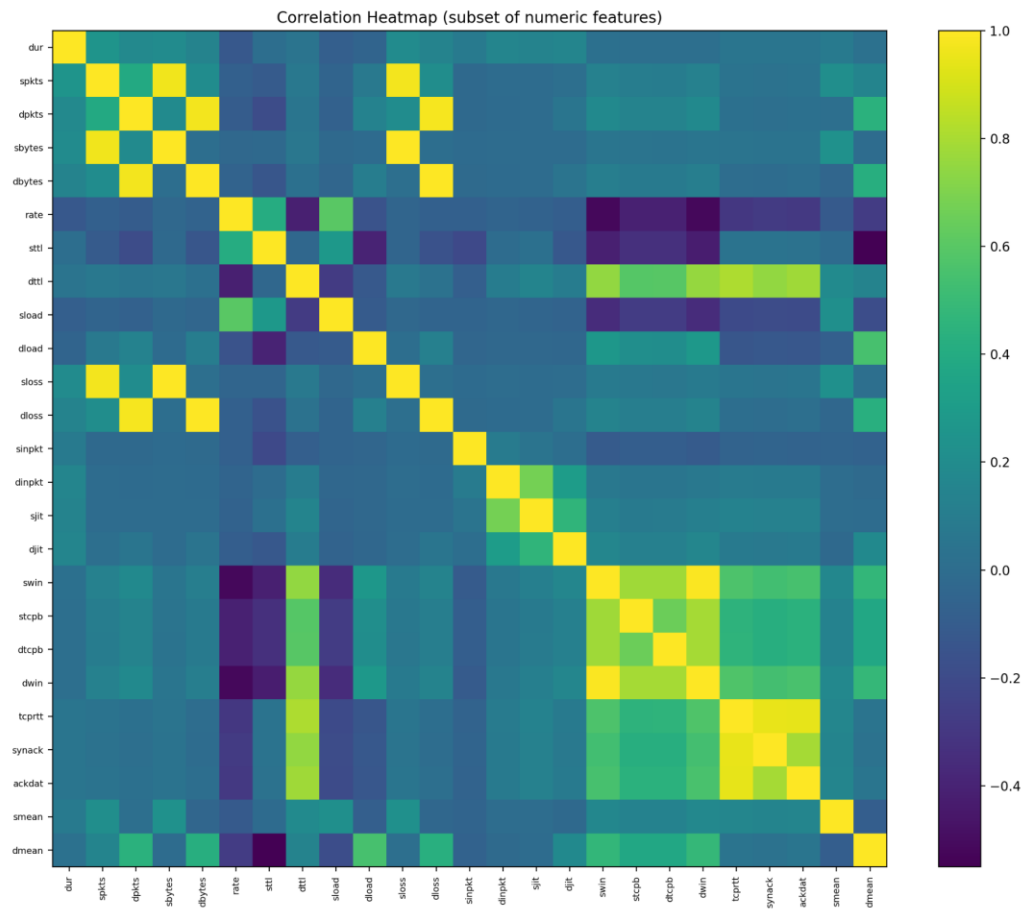


Figure 4: Correlation heatmap for a subset of numerical features

Unsupervised Anomaly Detection: Isolation Forest

Isolation Forest was employed as an unsupervised anomaly detection baseline. The model was trained exclusively on normal traffic samples in order to learn the typical distribution of normal network behavior. Anomaly scores were computed for the test set, and a detection threshold was defined as the 98th percentile of anomaly scores observed on the normal training data.

Figure 5 illustrates the distribution of anomaly scores for normal and attack traffic, along with the selected threshold.

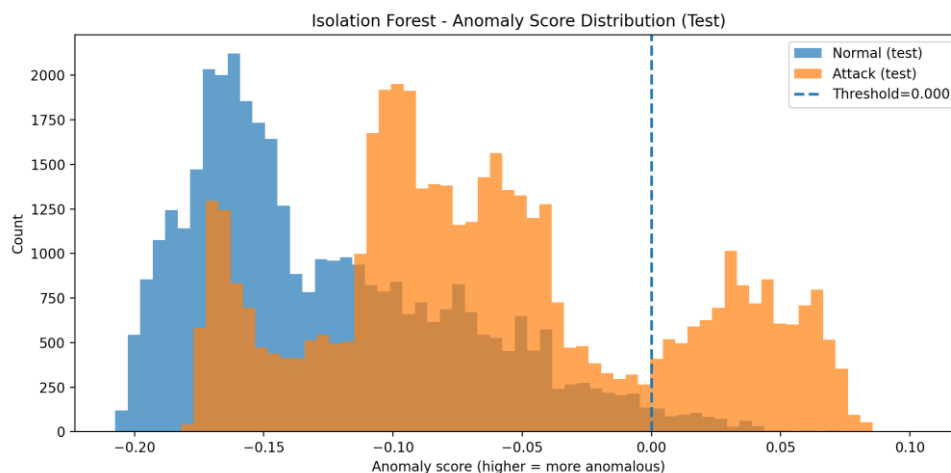


Figure 5: Isolation Forest anomaly score distribution on the test set with threshold

Supervised Intrusion Classification

Two supervised learning models were trained using labeled data: Logistic Regression and Random Forest. Logistic Regression was trained on SMOTE-balanced data to mitigate class imbalance, while Random Forest leveraged balanced sampling internally. These models learn explicit decision boundaries between normal and attack traffic.

Figure 6 compares the F1-scores obtained by the supervised models, demonstrating that Random Forest achieves the best overall performance due to its ability to capture non-linear feature interactions.

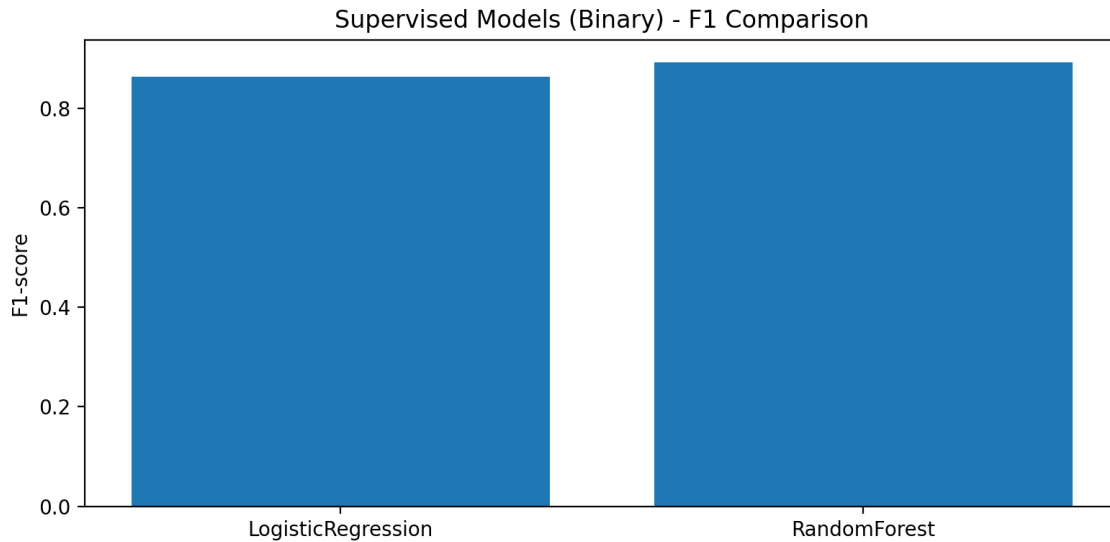


Figure 6: Supervised models (binary) – F1-score comparison

Deep Learning Anomaly Detection: Autoencoder

A deep autoencoder was implemented to model normal network traffic patterns in an unsupervised manner. The autoencoder was trained exclusively on normal traffic and optimized to minimize reconstruction error. During inference, reconstruction error (MSE) was used as an anomaly score. A detection threshold was defined as the 98th percentile of reconstruction errors observed on normal training data.

Figures 7–8 show reconstruction error distributions for normal and attack traffic. A zoomed view is included to improve readability when a small number of samples create a long right tail.

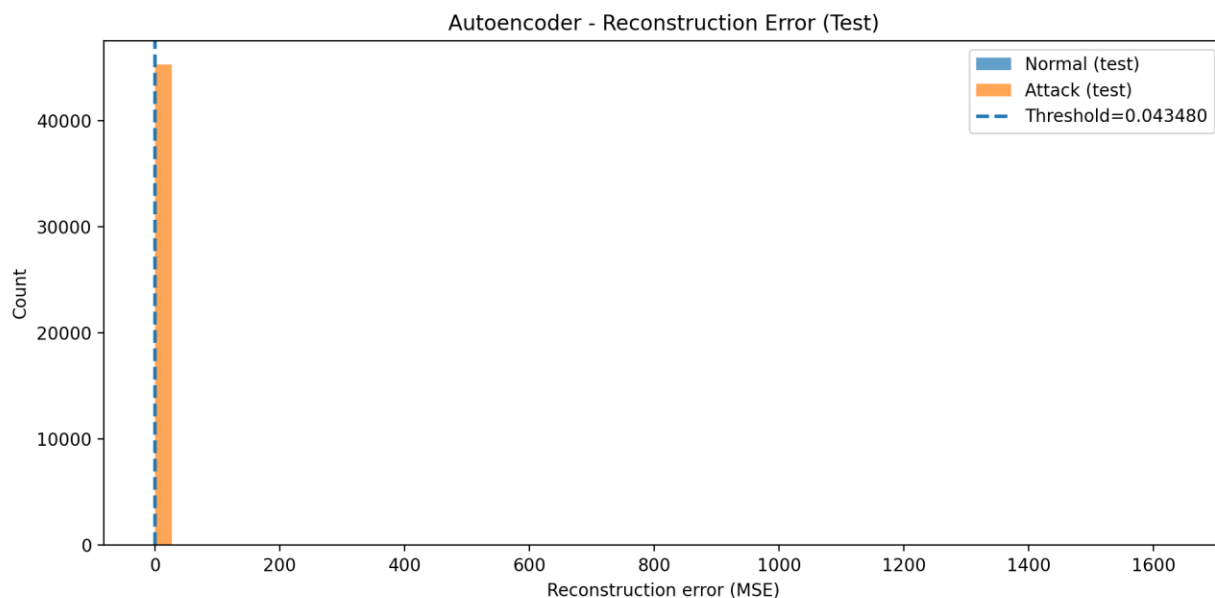


Figure 7: Autoencoder reconstruction error distribution on the test set with threshold

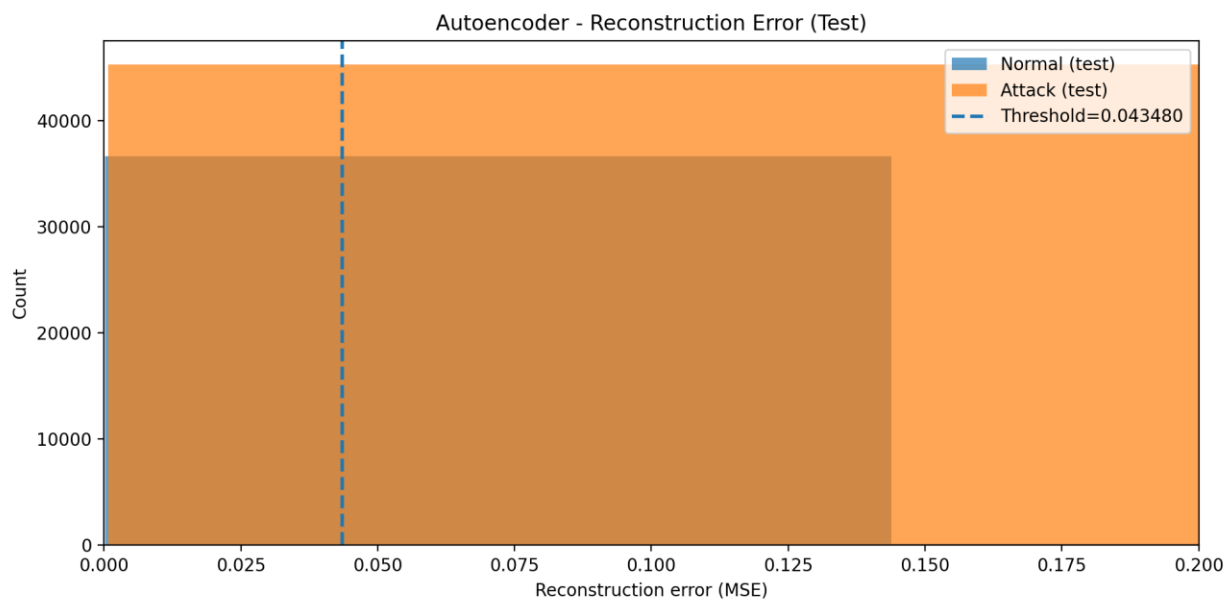


Figure 8: Autoencoder reconstruction error distribution (zoomed view) for readability

Evaluation

Models were evaluated using precision, recall, F1-score, confusion matrices, and ROC-AUC. The main results for binary classification are summarized below:

- Isolation Forest:	F1-score = 0.369, ROC-AUC = 0.777
- Autoencoder:	F1-score = 0.730, ROC-AUC = 0.891
- Logistic Regression:	F1-score = 0.863, ROC-AUC = 0.956
- Random Forest:	F1-score = 0.893, ROC-AUC = 0.980

Model Tuning

Model tuning focused on adjusting key hyperparameters for each approach in order to obtain stable and meaningful results.

For the autoencoder, the bottleneck size and number of training epochs were selected to balance reconstruction quality and generalization, ensuring that the model learned normal traffic patterns without simply memorizing the input. The detection threshold was derived from the reconstruction error distribution of normal traffic using a percentile-based rule, which helped control the trade-off between false positives and false negatives.

In the case of Isolation Forest, parameters such as contamination rate and number of estimators were chosen to reflect the expected proportion of anomalies.

Overall, tuning was aimed at improving detection reliability while avoiding overly aggressive anomaly labeling.

Model Selection

Model selection was based on a comparison between supervised and unsupervised approaches. When labeled data is available, supervised models achieved the best overall performance, as they can directly learn the distinction between normal and attack traffic.

Among the unsupervised methods, the autoencoder performed better than Isolation Forest because it captures complex relationships between features and detects deviations from learned normal behavior. Isolation Forest mainly identifies strongly isolated points,

which can reduce its ability to detect more subtle attacks. For both unsupervised models, thresholds were computed from normal-only score distributions, meaning that some normal samples exceeded the threshold and some attacks fell below it, especially when attack traffic closely resembled normal behavior.

Based on these observations, the autoencoder was preferred over Isolation Forest for unsupervised anomaly detection.

Conclusion

This project highlights the strengths and limitations of different machine learning approaches for intrusion detection. While supervised models achieve superior accuracy, unsupervised methods such as autoencoders offer greater adaptability to unseen attacks. Future work may include feature selection, improved threshold optimization, multi-class attack classification, and deployment in real-time monitoring systems.